

Publishing Method (GitHub Pages)

Table of contents

1	Goals	1
2	Definitions	2
3	Publishing Policy (Tag-based)	2
3.1	What triggers publication	2
3.2	What does NOT trigger publication	2
3.3	Why tags	2
4	Daily Editing Workflow	2
4.1	1) Work on branches	2
4.2	2) Preview locally while writing	2
4.3	3) Open a Pull Request (PR)	2
5	Previewing Changes (Branch / PR / Main)	2
5.1	Recommended preview methods	2
6	Release Workflow (How to publish)	3
6.1	0) Pre-release checklist	3
6.2	1) Release approval	3
6.3	2) Create the release tag	3
6.4	3) CI publishes GitHub Pages from the tag	3
7	Roles and Approvers	3
7.1	Approvers (must approve every release)	3
7.2	Release managers (may create tags / publish)	3
7.3	Optional strengthening (as we approach submission)	4
8	Versioning Guidance	4
9	Notes	4

This repository uses **Quarto** to render a documentation website to GitHub Pages.

We are adopting **Option B: tag-based publishing** to build release discipline ahead of 510(k) submission. Day-to-day edits can continue rapidly, but **public publication** is tied to explicit versioned releases.

1 Goals

- Enable fast iteration on content without continuously changing the public site.
- Ensure each public publication maps to a specific, immutable git state.
- Maintain a clear, auditable workflow identifying who approved and who performed a release.
- Provide practical ways for reviewers to preview changes before merging and before release.

2 Definitions

- **Source:** .qmd files and assets in this repository.
- **Rendered site:** output written to `_site/` by Quarto.
- **Release:** a published website update that corresponds to a git tag (e.g. `v0.3.0`).
- **Approver(s):** named individuals responsible for release approval.
- **Release manager:** the person who creates the tag (or GitHub Release) and triggers publication.

3 Publishing Policy (Tag-based)

3.1 What triggers publication

- The public GitHub Pages site should publish **only** from **version tags** on **main**, e.g.:
 - `v0.1.0`, `v0.2.0`, `v1.0.0`

3.2 What does NOT trigger publication

- Normal pushes to feature branches.
- Normal merges/pushes to **main** that are not tagged.

3.3 Why tags

- A tag is an immutable pointer to a specific commit, making it easy to answer:
 - “What exact content is currently published?”
 - “Which changes are in the published documentation vs in-progress edits?”

4 Daily Editing Workflow

4.1 1) Work on branches

- Create a feature branch for each change set.
- Keep PRs small when possible (one theme per PR).

4.2 2) Preview locally while writing

- Use Quarto’s preview server:
 - `quarto preview`
- Use a full render to validate a clean build:
 - `quarto render`

4.3 3) Open a Pull Request (PR)

- Use PR review to gate changes into **main**.
- Prefer requiring at least one reviewer for all PRs.

5 Previewing Changes (Branch / PR / Main)

We need reviewers to preview changes before merging and before releasing.

5.1 Recommended preview methods

- 1) **Local preview (fastest for authors)**
 - Authors run `quarto preview` while editing.
 - Good for rapid iteration, but not ideal as the only review mechanism.

2) **PR build artifacts (recommended for reviewers)**

- CI renders the site for a PR and uploads `_site/` as a downloadable artifact.
- Reviewers download and open the generated HTML locally (or serve it locally).
- Benefits:
 - Preview is reproducible (built in CI).
 - No need for reviewers to install Quarto/TinyTeX.

3) **Main preview**

- `main` can be previewed the same way as PRs:
 - CI builds and attaches artifacts for commits.
 - Or a maintainer can run `quarto preview` locally.

Note: GitHub Pages itself is reserved for **released tags**, so it remains stable and clearly “official.”

6 Release Workflow (How to publish)

6.1 0) Pre-release checklist

- PRs merged to `main` are reviewed and CI is green.
- Any visible version string in the site (e.g., footer “Document Version”) is updated as appropriate.
- Decide the next tag (semantic versioning recommended).

6.2 1) Release approval

- A release is approved by the designated approver(s) listed below.
- Approval should be recorded in one of these forms:
 - A GitHub Issue/Discussion titled “Release vX.Y.Z approval” with explicit approval comments.
 - A PR that bumps the version, with required reviewers approving.

6.3 2) Create the release tag

- The release manager creates an annotated tag on the intended commit on `main`:
 - `vX.Y.Z`

6.4 3) CI publishes GitHub Pages from the tag

- GitHub Actions renders and deploys the site.
- The published site now corresponds to `vX.Y.Z`.

7 Roles and Approvers

7.1 Approvers (must approve every release)

Fill these in and keep them current.

- Primary approver: TBD
- Secondary approver (backup): TBD

7.2 Release managers (may create tags / publish)

Fill these in and keep them current.

- Release manager(s): TBD

7.3 Optional strengthening (as we approach submission)

- Add CODEOWNERS so specific files/sections require review.
- Require branch protection on `main`:
 - required reviews
 - required status checks (Quarto render)
- Add a short “release PR” that bumps the displayed document version and links the release tag.

8 Versioning Guidance

- Use semantic-ish versioning even during early development:
 - `v0.MINOR.PATCH` is fine for pre-1.0 work.
- Tie the site-visible “Document Version” to the tag name whenever possible.

9 Notes

- `_site/` is build output and should not be hand-edited.
- Stopping `quarto preview` with Ctrl-C results in a non-zero exit code (expected).